



OBJECT ORIENTED ANALYSIS AND DESIGN

Ch - 5

Advanced Use case Modeling

A study material for the students of GLS University

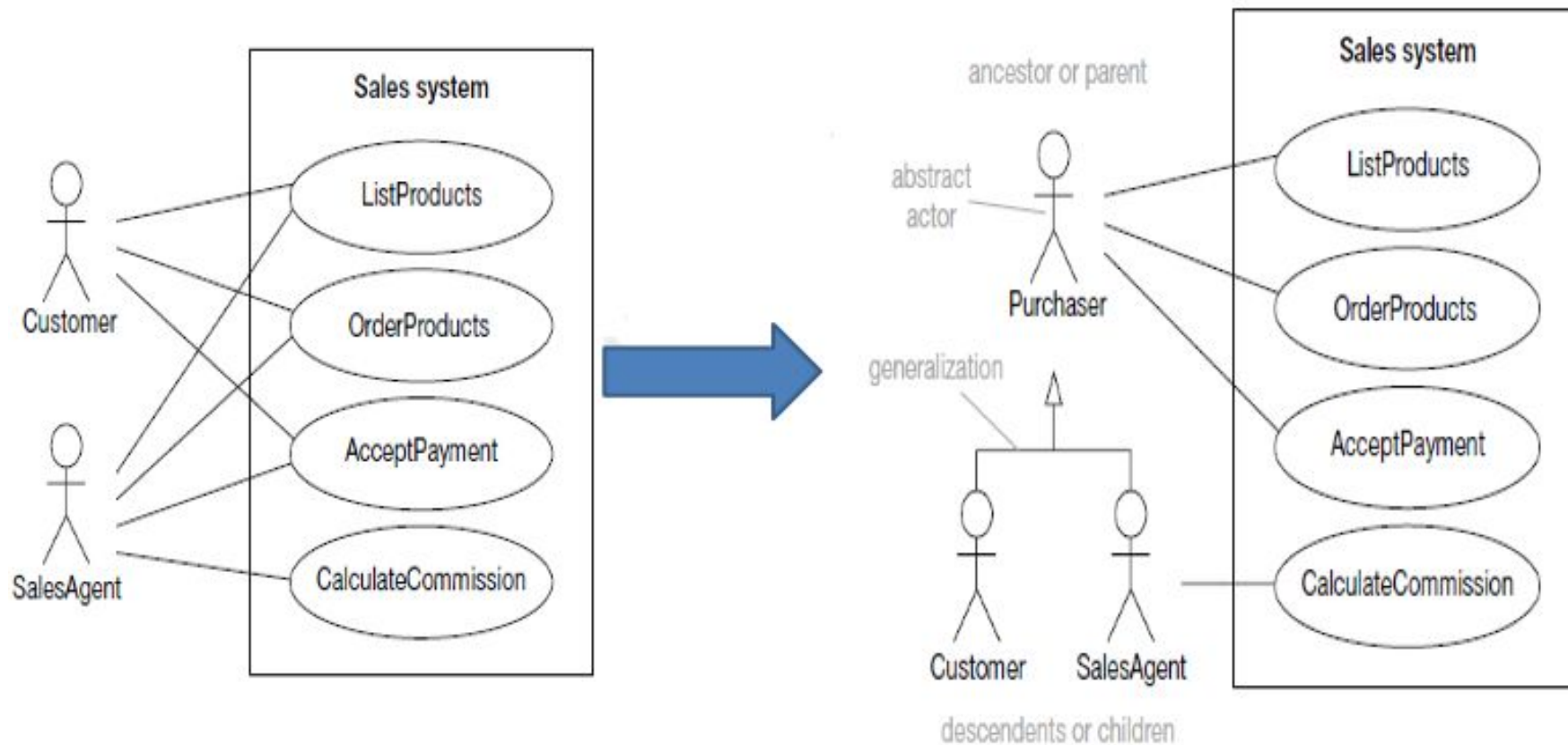
Actor Generalization

Actor generalization is the process of defining a parent actor (super actor) that holds common interactions, and child actors that inherit those behaviors and may also have additional, specific use cases.

Why Use Actor Generalization?

- To reuse common interactions.
- To simplify complex use case diagrams.
- To represent hierarchies like User → Admin/User/Guest.
- To promote clarity and maintainability.

Example



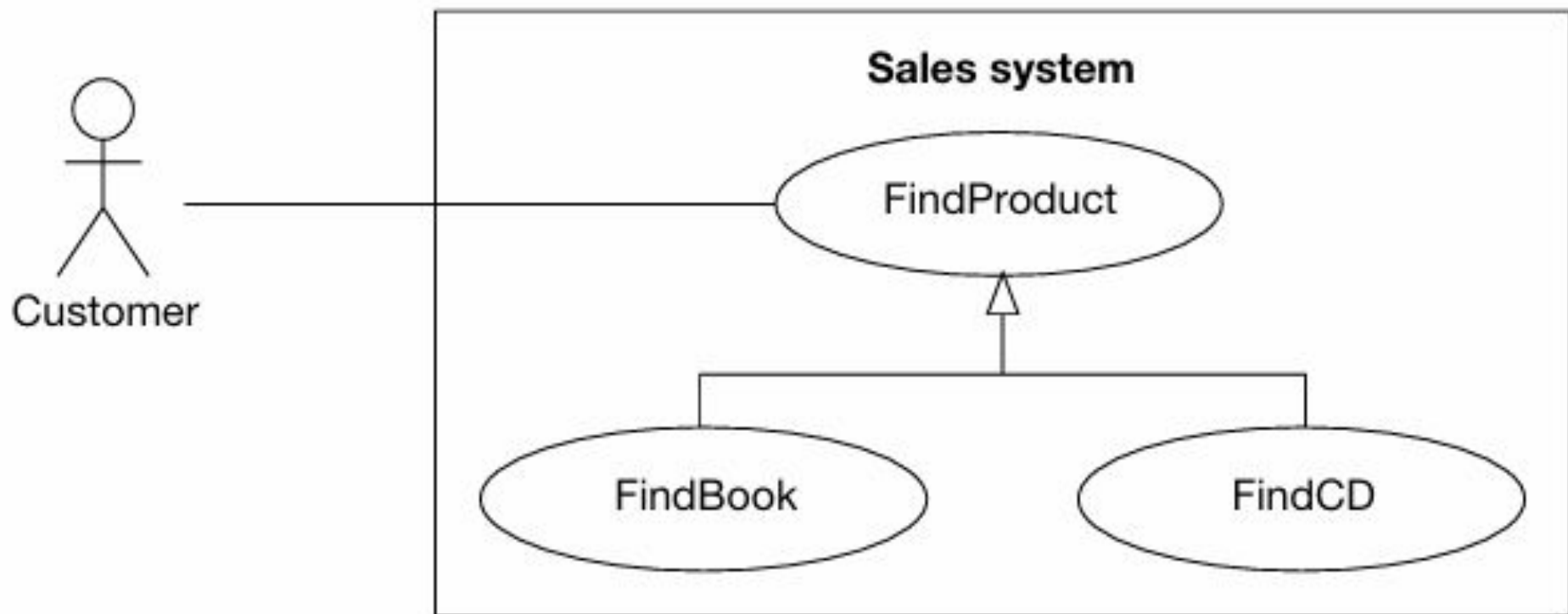
Use case Generalization

Use case generalization allows you to:

- Factor out common behavior from two or more similar use cases.
- Place the shared logic in a parent use case.
- Let child use cases inherit and extend or override some parts of the parent behavior.

Why Use It?

- Avoids redundancy in use case descriptions.
- Models polymorphism in user interactions.
- Helps if different scenarios share a basic structure, but have some variations.



<<include>> Relationship

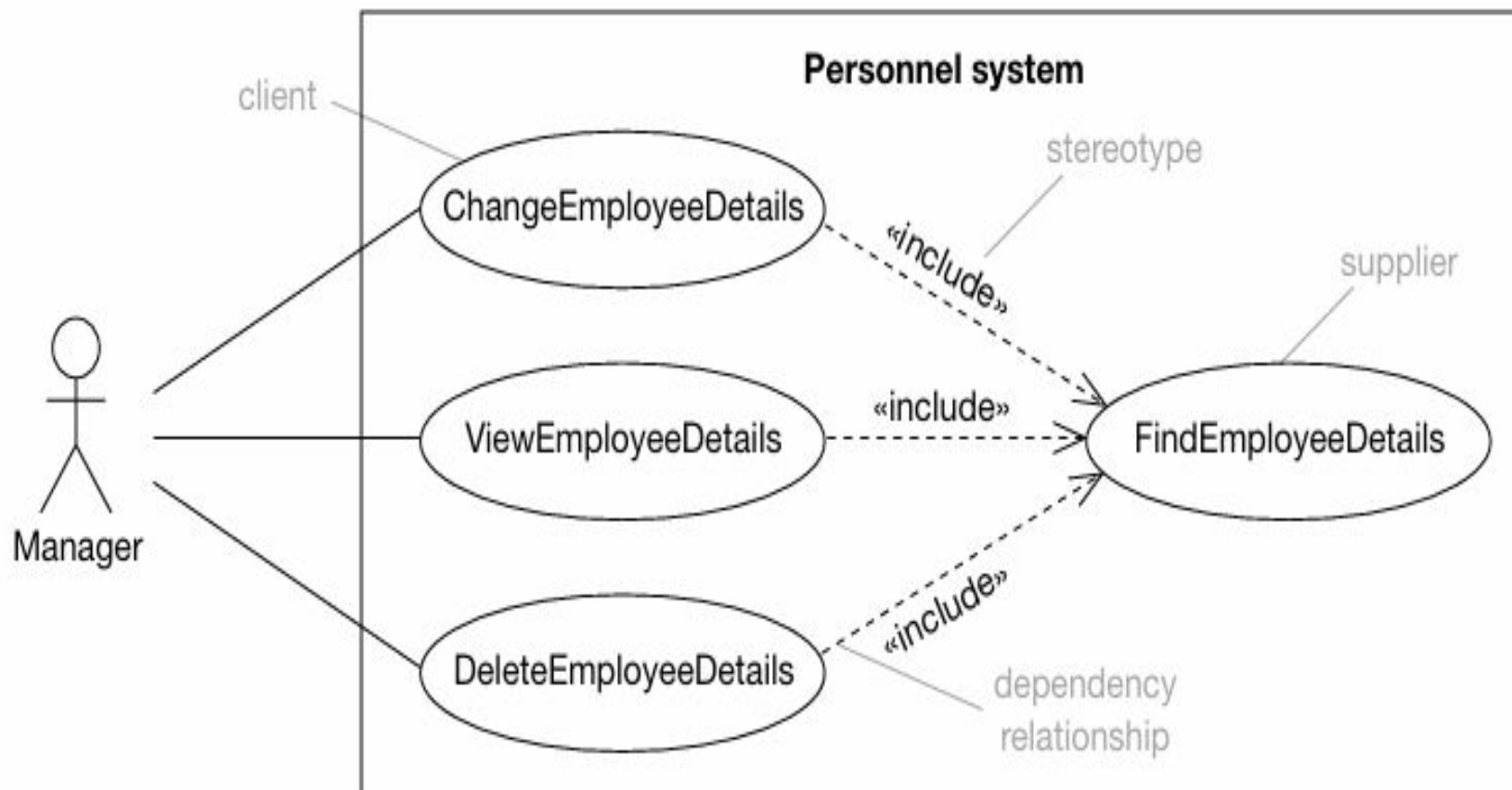
The «include» relationship extracts common behavior that is shared between multiple use cases, and puts it into a separate reusable use case.

Why use «include»?

When multiple use cases have repetitive steps, instead of writing the same thing again and again, you:

- Extract those common steps into a new use case.
- Use the «include» relationship to include that new use case wherever needed.

<<include>>



<<extend>> Relationship

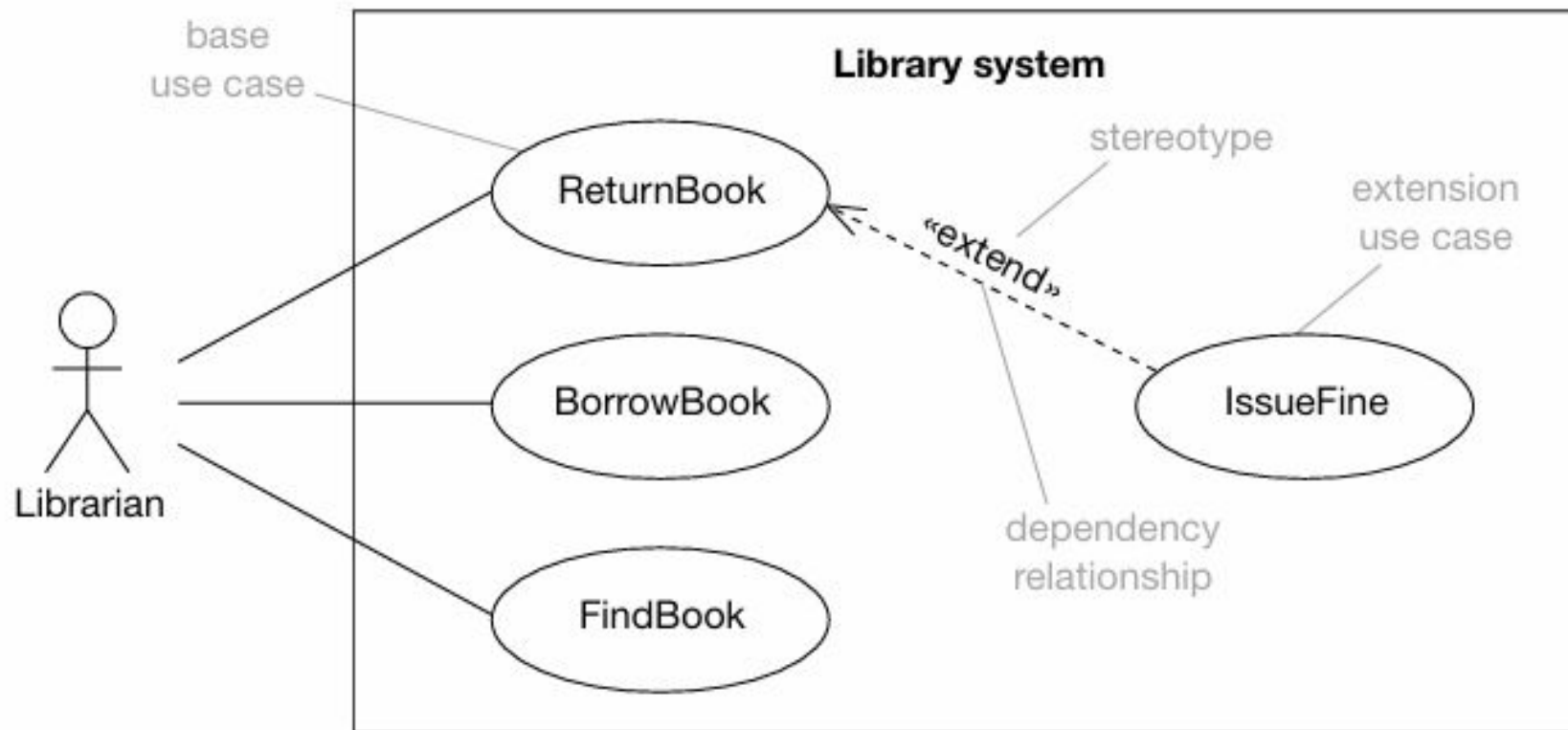
The «extend» relationship is used when a use case optionally adds behavior to another use case under certain conditions.

Why use «extend»?

Use «extend» when you have a base use case, and sometimes, depending on a condition or situation, you want to extend its behavior.

The extended part is not always executed — only when a specific condition is true.

<<extend>>



Case Study - Online Food Delivery System

Part 1 — Understand the System

define the system boundary.

System -

Online Food Delivery System is responsible for:

- Customer management
- Restaurant management
- Order management
- Payment management
- Delivery management
- Offers and coupons
- Reviews and ratings
- Notifications

Part 2 — Finding Actors

- Customer
- Restaurant
- Delivery Partner
- Payment Gateway
- Notification Service

Case Study - Online Food Delivery System

Part 3 — Identify Basic Use Cases

Customer -

- Register
- Login
- Search Restaurant
- View Menu
- Place Order
- Make Payment
- Track Order
- Cancel Order
- Apply Coupon
- Rate Order

Restaurant Staff -

- Manage Menu
- View Order
- Accept Order
- Reject Order
- Update Order Status

Delivery Partner -

- View Delivery
- Accept Delivery
- Update Delivery Status
- Confirm Delivery

Payment Gateway -

- Process Payment

Notification Service -

- Send Notification

Case Study - Online Food Delivery System

Part 4 — Actor Generalization

Regular Customer

- Search restaurants
- Place orders
- Make payments
- Track orders

Premium Customer

Can do everything a regular customer can do, plus:

- Get priority delivery
- Receive special offers
- Get premium discounts

Another possible Actor Generalization

Suppose the system has:

- Delivery Partner
- Delivery Supervisor

A Delivery Supervisor can perform all activities of a Delivery Partner, plus supervise multiple deliveries.

Case Study - Online Food Delivery System

Part 5 — Use Case Generalization

The general activity is: Make Payment

But payment can be performed through:

- Card Payment
- UPI Payment
- Cash Payment

Part 6 — Another Use Case Generalization

Suppose the customer can communicate with the restaurant in two ways:

- Send Text Message
- Send Voice Message

Both are specialized forms of: Send Message

Case Study - Online Food Delivery System

Part 7 — Finding <<include>>

When the customer makes an online payment, is authentication required.

Make Payment - - - - -> Authenticate Payment
<<include>>

Place Order

Before the system confirms an order, it must: Calculate Order Total

Place Order - - - - -> Calculate Order Total
<<include>>

Before confirming an order, the system must check whether the restaurant can accept the order.

Place Order - - - - -> Check Restaurant Availability
<<include>>

Case Study - Online Food Delivery System

Part 8 — Finding <<extend>>

Place Order

Sometimes the customer applies a coupon.

But does every customer use a coupon?

No.

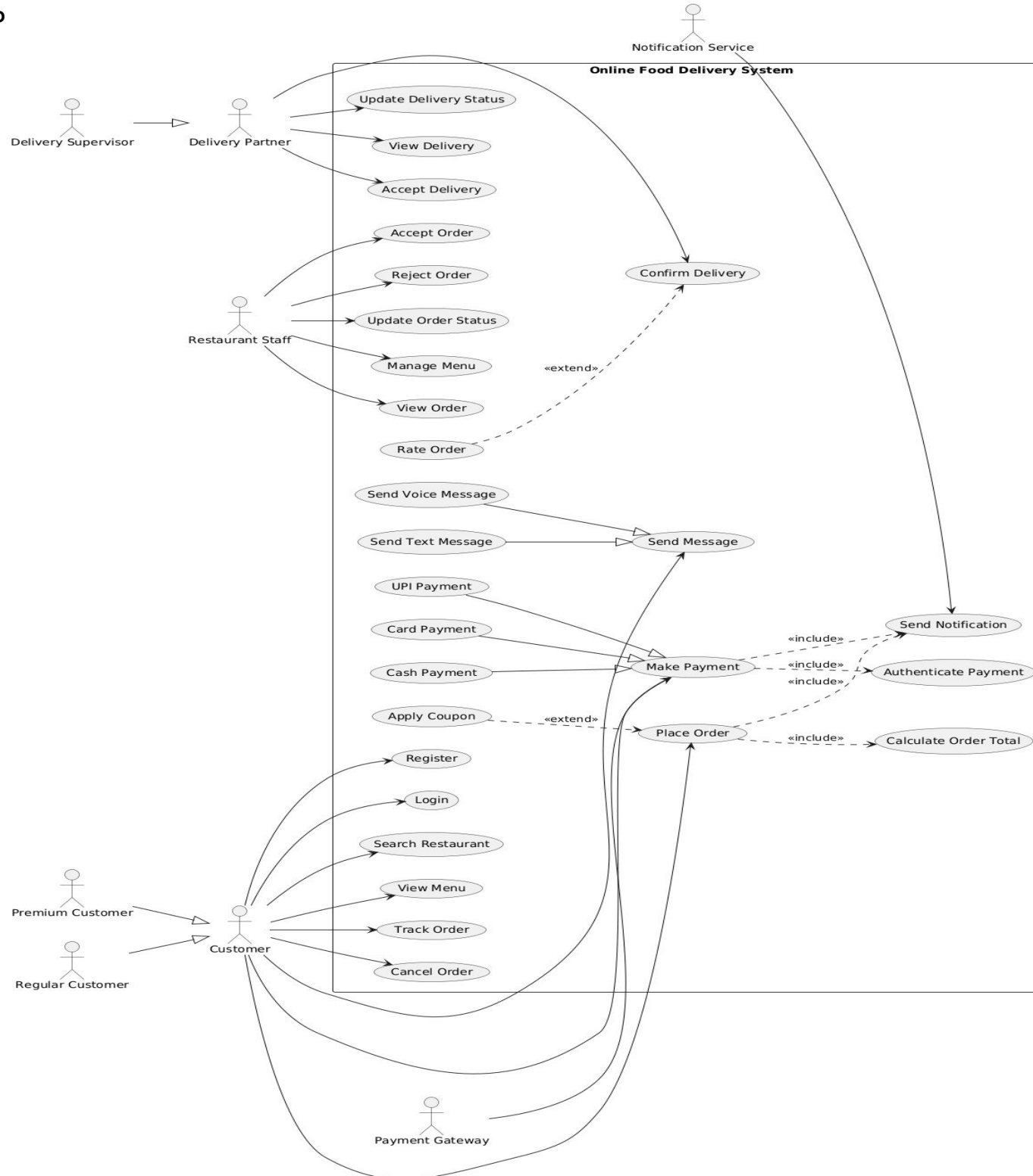
Apply Coupon - - - - - - - - - - -> Place Order
<<extend>>

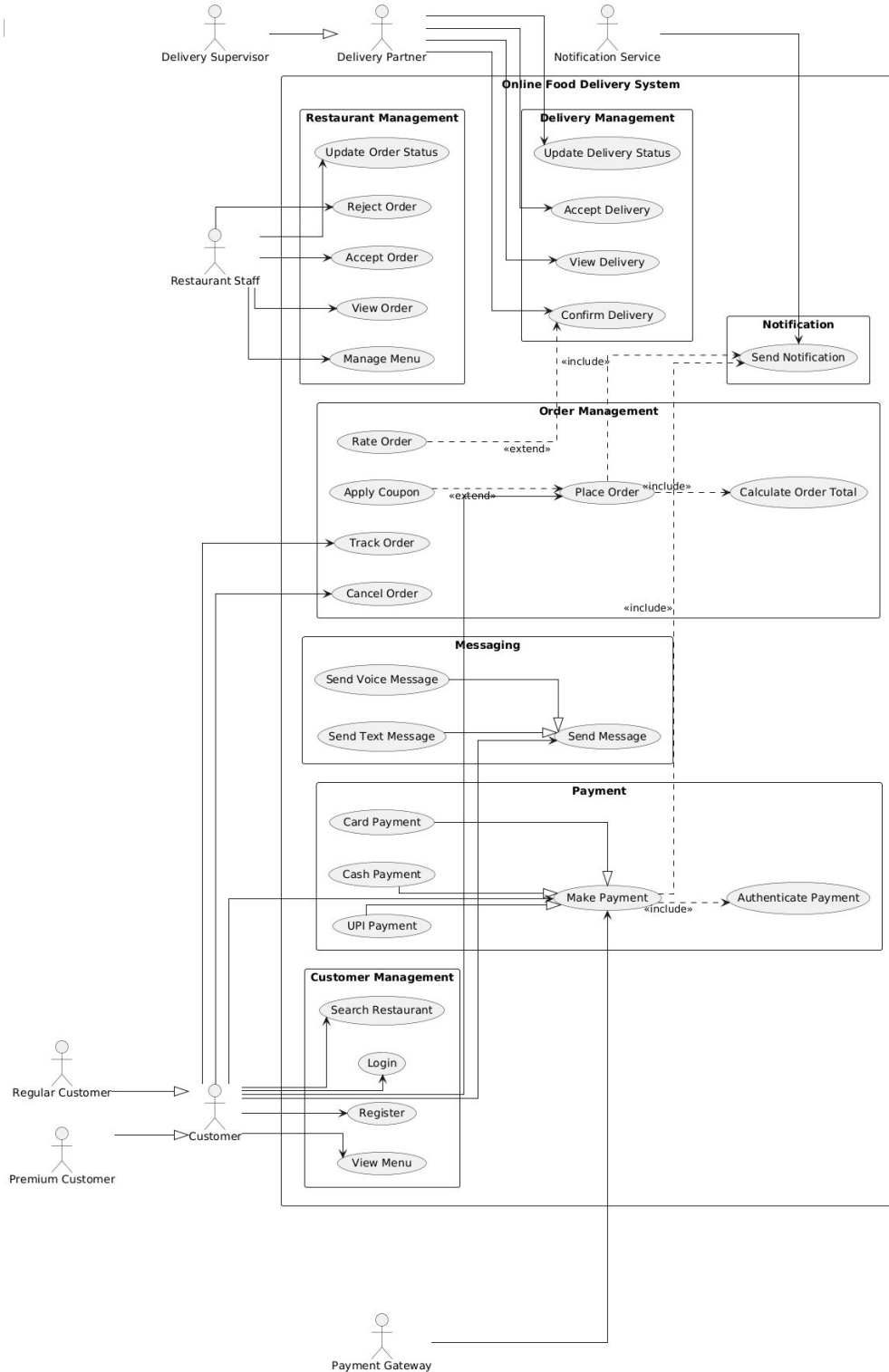
Part 9 — Another extend Example

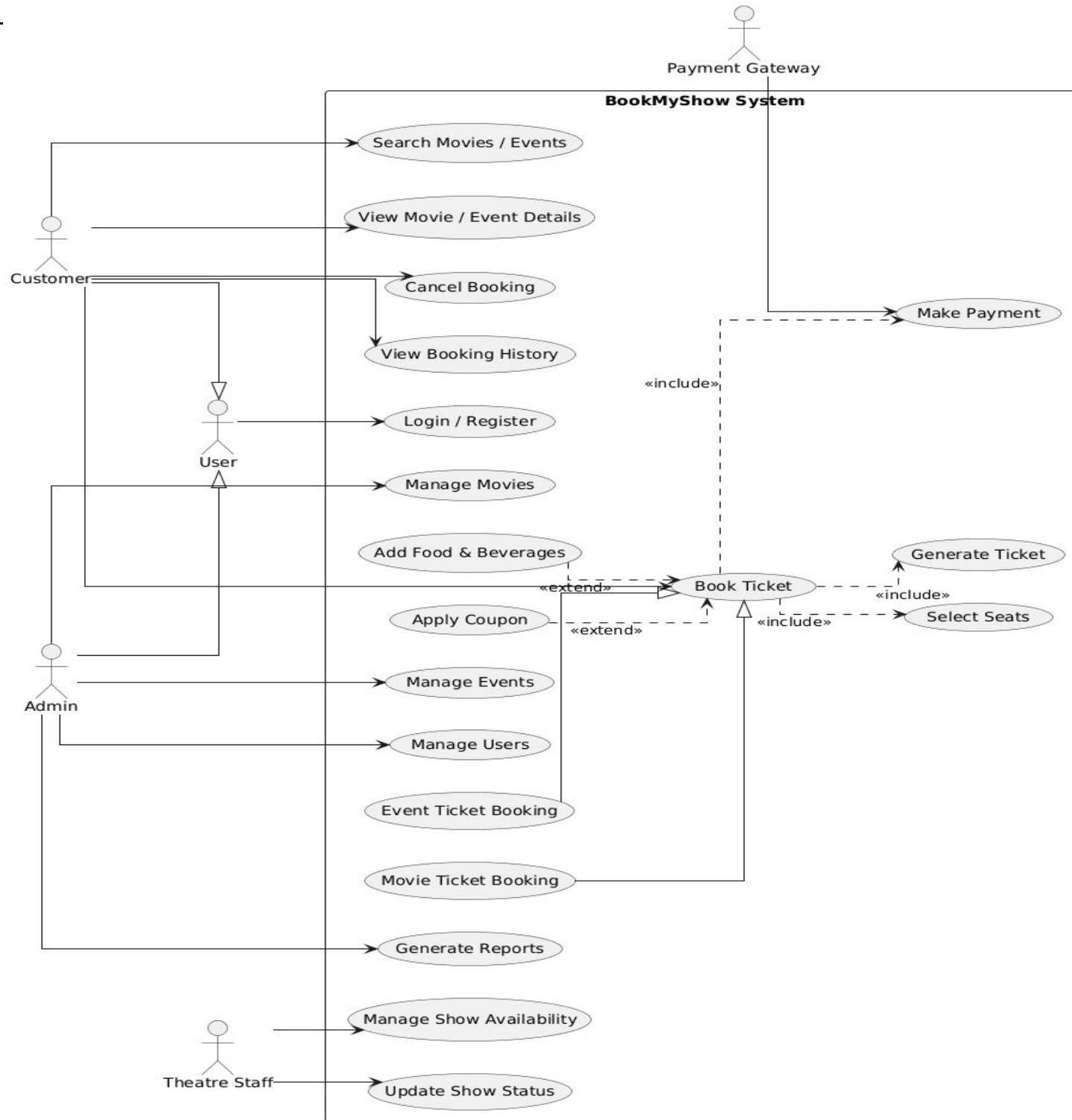
After an order is delivered, the customer may rate the restaurant.

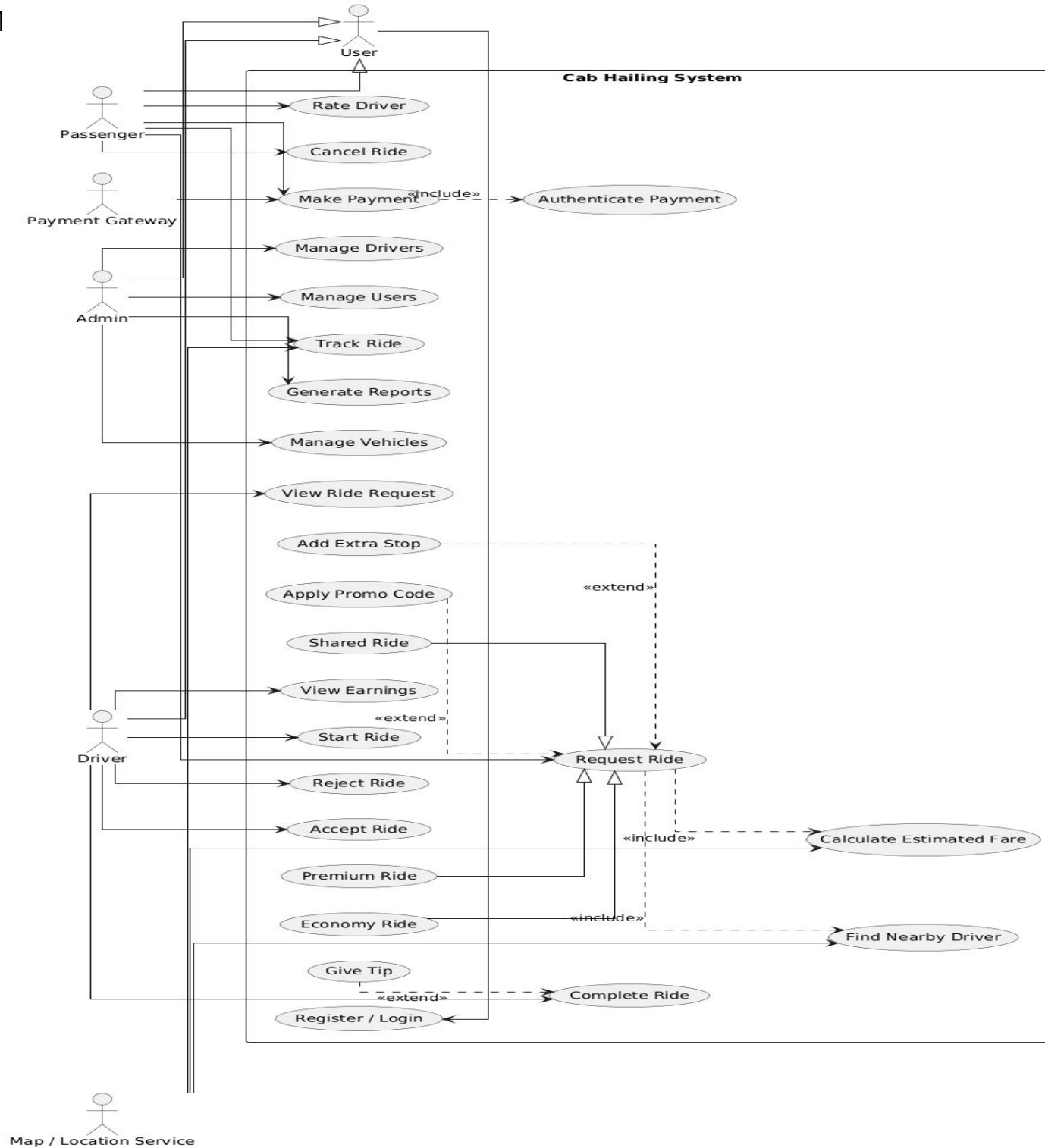
Not every customer has to provide a rating.

Rate Order - - - - - - - - - - -> Complete Order
<<extend>>









Case Study - University Course Registration

Design a UML Use Case Diagram for a University Course Registration System with the following requirements:

- A university course registration system is used by Students, Faculty Members, Department Heads, and Administrators.
- A Student can Search Courses, Register Course, Drop Course, View Timetable, View Results, and Add Course to Wishlist.
- Students are specialized into Undergraduate Student and Postgraduate Student.
- A Faculty Member can View Courses, Manage Attendance, and Upload Marks.
- A Department Head is a specialized Faculty Member and can additionally Approve Course Registration.
- An Administrator can Manage Students, Faculty, and Courses.
- During Register Course, the system must Check Prerequisites, Check Seat Availability, and Validate Student Eligibility.
- A student may optionally Add Course to Wishlist before registration.
- Identify and model suitable Use Case Generalization among related use cases.

Your diagram must clearly demonstrate:

- Actor Generalization
- Use Case Generalization
- <<include>> relationships
- <<extend>> relationship
- Correct system boundary and actor associations.

THANK YOU